HGS — Hybrid Genetic Search (Vidal 2022, Algorithm 1) with VRPP adaptations

Giant-tour genotype decoded by a profit-aware Linear Split; dual feasible / infeasible sub-populations; RP-GPX crossover; education by granular local search (SWAP*, 2-opt*, relocate…); biased fitness = profit rank + diversity rank; adaptive capacity penalty. Objective: maximise profit = R·Σ waste − C·Σ distance.

START
RouteConstructionAction → HGSPolicy.execute(context)

0. Policy adapter & dispatcher — HGSPolicy ("hgs") → run_hgs()

mandatory bins empty
(and not vrpp) ?

no

yes

Load capacity, R, C · sub-problem (all bins if vrpp)
HGSParams: time_limit 60 s, restart_timer, μ=25, λ=40 offspring,
nb_elite 4, nb_close 5, nb_granular 20, target_feasible 0.2,
n_iterations_no_improvement 20000, penalty 1.0 (×1.2 / ×0.85),
crossover_rate 1.0, repair_probability 0.5, max_vehicles, seed

return tour [0, 0]

engine ?

pyvrp

custom

≤ 2 nodes
(depot + 1 bin) ?

solve_pyvrp()
(PyVRP HGS)

no

yes

HGSSolver(dist, wastes, capacity, R, C, params,
mandatory, x/y coords).solve()

trivial answer:
[[1]] if it fits, else []

map local routes → global tour · km on full matrix

END
tour · profit · km

expands to

1. Solver set-up

LinearSplit(dist, wastes, capacity, R, C, mandatory, max_vehicles)
O(n) Bellman split with profit objective (see 5.)

HGSLocalSearch: granular neighbour lists (nb_granular nearest)
moves: relocate · swap · SWAP* · 2-opt · 2-opt* · Or-opt · cross
+ optional 3-opt, cross-exchange, λ-interchange, ejection chains

2. Main loop — HGSSolver.solve()

INITIALISE POPULATION: 4μ random giant tours
evaluate → LS educate → evaluate
insert into feasible / infeasible sub-population by capacity violation

best = max profit in feasible pop (or none)
it = 0 · it_no_improvement = 0 · penalty = initial

while True:

time_limit reached
or (no time limit and
it_no_improvement ≥ n_iterations_no_improvement) ?

no

yes

time-limited and
it_no_improvement ≥ restart_timer ?

yes

no

RESTART: reset penalty & caches,
re-initialise population, it_no_improvement = 0

yes

|feasible| + |infeasible| < 2 ?

no

yes

it += 1 · it_no_improvement += 1
child = generate_offspring() (see 3.)

return best feasible (routes, profit, cost)
or best of both pops if no feasible found

expands to

insert_and_repair(child) (see 4.)

expands to

child feasible and
profit > best ?

yes

best ← child · it_no_improvement = 0

no

a sub-population reached
μ + λ individuals ?

yes

no

SURVIVOR SELECTION (trim to μ):
update biased fitness; repeatedly remove a clone
(broken-pairs distance 0) else the worst-fitness individual

it mod 100 == 0 ?

yes

no

ADAPT PENALTY
CVRP: feasible ratio vs target_feasible ± 0.05 → ×0.85 / ×1.2
VRPP: offspring coverage & profit margin signals → ×0.85 / ×1.2 / ×√1.2,
clamped to [min, max]

criterion.step(…) · _viz_record(it, best, child, pop size)

next

3. Offspring generation — _generate_offspring()

PARENT SELECTION: two binary tournaments over the union
of both sub-populations (lower biased fitness wins)

rand < crossover_rate ?

yes

no

RP-GPX crossover (route-based profit-aware Generalized Partition):
physical edges of p1 ∪ p2 → uncommon-edge components →
inherit best component per partition → pack remaining nodes
(mandatory always kept, capacity respected)

child = clone of p1 (mutation-only step)

evaluate(child): active sequence → Split → routes,
profit, cost, revenue, capacity violation, penalised cost

EDUCATE: child = HGSLocalSearch.optimize(child)
first-improvement over granular (u, v) pairs until no move improves
penalised profit; may insert unvisited profitable nodes or drop unprofitable ones;
giant tour rebuilt with unvisited nodes interleaved proportionally

evaluate(child) again → return child

4. Insertion & repair — _insert_and_repair(child)

insert child into feasible or infeasible sub-population
(by its own capacity violation) · record offspring signals
(feasibility, coverage, margin) for penalty adaptation

not vrpp and child infeasible
and rand < repair_probability ?

yes

REPAIR: re-evaluate with penalty × 10 → LS educate →
re-evaluate with normal penalty → insert repaired copy

5. Decoding — evaluate() + LinearSplit.split(active_sequence)

active sequence = giant-tour order restricted to nodes
in current routes (or, first time, nodes with R·w ≥ C·(d₀ᵢ + dᵢ₀) or mandatory)

prefix sums of load, revenue, distance
max_vehicles = 0 → unlimited split (monotone deque, O(n))
else limited split (k-vehicle DP) · fallback greedy split if infeasible

routes, profit → cost, revenue, capacity violation,
is_feasible, penalised cost = cost + penalty · violation

6. Biased fitness — update_biased_fitness(pop, nb_elite, nb_close)

rank_profit: sort by penalised profit (desc)
rank_diversity: mean broken-pairs distance to nb_close nearest (desc)
fitness = rank_profit + (1 − nb_elite / |pop|) · rank_diversity (lower is better)